

COIMBATORE INSTITUTE OF TECHNOLOGY

*(Government Aided Autonomous Institution Approved by AICTE, New Delhi &
Affiliated to Anna University, Chennai)*



Department of Computing

Information Security Computing Laboratory

(20MSS94)

Title: SecureLab - Web Security Attack Simulations

Rithanyaa M E

71762231042

M.Sc., Software Systems (9th Semester)

71762231042@cit.edu.in

Contents

1 Abstract	2
2 Introduction	2
3 Technology Stack	2
4 System Architecture	2
4.1 Database Schema	3
5 Attack 1 — SQL Injection	4
5.1 Concept	4
5.2 Real-World Impact	4
5.3 Scenario	4
5.4 Vulnerable Code	4
5.5 Attack Payload and Execution	4
5.6 Secure Code	5
5.7 Comparison	5
5.8 Prevention Checklist	5
6 Attack 2 — Cross-Site Scripting (XSS)	5
6.1 Concept	5
6.2 Real-World Impact	5
6.3 Scenario	6
6.4 Vulnerable Code	6
6.5 Attack Payloads	6
6.6 Secure Code	6
6.7 Comparison	7
6.8 Prevention Checklist	7
7 Attack 3 — Parameter Tampering	7
7.1 Concept	7
7.2 Real-World Impact	7
7.3 Scenario	7
7.4 Vulnerable Code	8
7.5 Attack Steps	8
7.6 Secure Code	8
7.7 Comparison	8
7.8 Prevention Checklist	9
8 Application Features	9
9 Results and Observations	9
10 Conclusion	10
11 References	10

1 Abstract

This project demonstrates three critical web application security vulnerabilities through a fully interactive simulation platform called **SecureLab**. Built using Python Flask, SQLite, and HTML/CSS/JavaScript, the application allows users to execute live attacks — SQL Injection, Cross-Site Scripting (XSS), and Parameter Tampering — in a controlled environment. Each attack is paired with a secure implementation, illustrating the exact fix required. The project covers all evaluation aspects: simulation, rectification, design, viva, and documentation.

2 Introduction

Web applications form the backbone of modern digital infrastructure — from banking to e-commerce to social media. Poorly coded applications expose millions of users to attackers who exploit subtle but catastrophic flaws in how user input is handled.

The **OWASP Top 10** (Open Web Application Security Project) is the industry-standard list of the most critical web security risks. Three of the most impactful attacks covered in this project are:

1. **SQL Injection** — OWASP A03:2021 Injection
2. **Cross-Site Scripting (XSS)** — OWASP A03:2021 Injection
3. **Parameter Tampering** — OWASP A04:2021 Insecure Design

Key Principle: “*Never trust user input.*” — The golden rule of web security.

3 Technology Stack

Layer	Technology	Version	Purpose
Backend	Python + Flask	3.1.0	Route handling, business logic
Database	SQLite	3.x	Users, comments, products, orders
Frontend	HTML5 + CSS3	—	Minimal light-mode UI
Scripting	Vanilla JavaScript	ES6+	AJAX calls, dynamic updates
Templating	Jinja2	—	Server-side HTML rendering
Fonts	Inter + JetBrains Mono	—	Typography and code blocks

Why Flask? Flask is a lightweight Python web framework widely used for security demonstrations. Its minimal overhead makes raw SQL queries and parameter handling visible without framework-level abstractions hiding the vulnerability.

4 System Architecture

```
CAT1_Project/
├─ app.py           – Flask application (all routes, attack logic)
├─ database.py      – SQLite initialization and seed data
```

- └─ demo.db ─ SQLite database (auto-generated on startup)
- └─ requirements.txt ─ Python dependencies
- └─ templates/
 - └─ base.html ─ Shared layout (navbar, footer)
 - └─ index.html ─ Home dashboard with 3 attack cards
 - └─ sql_injection.html ─ SQL Injection demo page
 - └─ xss.html ─ XSS demo page
 - └─ param_tamper.html ─ Parameter Tampering demo page
- └─ static/
 - └─ css/style.css ─ Minimal light-mode UI theme
 - └─ js/main.js ─ Shared utilities

4.1 Database Schema

users table (SQL Injection demo):

```
CREATE TABLE users (
  id      INTEGER PRIMARY KEY AUTOINCREMENT,
  username TEXT NOT NULL,
  password TEXT NOT NULL,
  role    TEXT DEFAULT 'user',
  balance REAL DEFAULT 0.0
);
```

username	password	role	balance
admin	SuperSecret@123	admin	Rs. 98,500
alice	alice_pass	user	Rs. 1,250
bob	bob_pass	user	Rs. 320

comments table (XSS demo):

```
CREATE TABLE comments (
  id      INTEGER PRIMARY KEY AUTOINCREMENT,
  author  TEXT NOT NULL,
  content TEXT NOT NULL,
  timestamp TEXT DEFAULT (datetime('now','localtime'))
);
```

products and orders tables (Parameter Tampering demo):

```
CREATE TABLE products (id, name, price REAL, stock INTEGER);
CREATE TABLE orders  (id, product_name, real_price REAL,
                        paid_price REAL, tampered INTEGER, timestamp TEXT);
```

5 Attack 1 — SQL Injection

5.1 Concept

SQL Injection (SQLi) is a technique that exploits vulnerabilities in web applications by inserting malicious SQL statements into input fields. When user input is directly concatenated into a SQL query string, the attacker can alter the query's logic entirely — bypassing authentication, extracting data, or destroying the database.

5.2 Real-World Impact

- **Heartland Payment Systems (2008)** — 134 million credit card records stolen via SQL Injection
- **Sony Pictures (2011)** — 1 million user records exposed
- **OWASP Ranking:** A03:2021 Injection

5.3 Scenario

A fake banking login page (FakeBank) is used. The database contains an admin user with a secret password. The attacker's goal is to log in as admin **without knowing the password**.

5.4 Vulnerable Code

```
# app.py - /sql-injection/vulnerable (DELIBERATELY INSECURE)
username = request.form.get('username', '')
password = request.form.get('password', '')

# VULNERABLE: raw string concatenation - never do this!
query = f"SELECT * FROM users WHERE username='{username}' AND password='{password}'"
result = conn.execute(query).fetchone()
```

5.5 Attack Payload and Execution

Attacker enters **username:** admin' -- with any password.

Query the database actually executes:

```
SELECT * FROM users
WHERE username='admin' --' AND password='anything'
-- Everything after -- is a COMMENT → password check skipped!
```

The -- is a SQL comment delimiter. The password check is entirely ignored. The database returns the admin row, granting full access without the correct password.

Other payloads:

```
' OR '1'='1      → Always-true condition, returns first user
admin' OR 1=1 --  → Targets admin, bypasses password check
```

5.6 Secure Code

```
# app.py - /sql-injection/secure (FIXED)
username = request.form.get('username', '')
password = request.form.get('password', '')

# SECURE: parameterized query - values bound separately
query = "SELECT * FROM users WHERE username = ? AND password = ?"
result = conn.execute(query, (username, password)).fetchone()
# Input is treated purely as data - never as SQL code
```

5.7 Comparison

	Vulnerable	Secure
Method	Raw string concatenation	Parameterized query (?)
Input role	Becomes SQL code	Treated as a plain value
admin' --	Bypasses login	Fails — no such username

5.8 Prevention Checklist

- Always use parameterized queries or prepared statements
- Use ORMs (SQLAlchemy, Django ORM) which parameterize by default
- Apply principle of least privilege to the database user account
- Validate and whitelist input where possible
- **Never** concatenate user input directly into SQL strings

6 Attack 2 — Cross-Site Scripting (XSS)

6.1 Concept

XSS occurs when an attacker injects malicious JavaScript into web pages viewed by other users. The injected script runs in the victim's browser with the same trust level as the legitimate site giving the attacker access to cookies, session tokens, and the ability to manipulate page content.

Type	Description
Stored XSS (<i>demonstrated</i>)	Script saved to DB, executes for every visitor
Reflected XSS	Script in URL, executes when link is opened
DOM-based XSS	Script manipulates DOM via client-side JavaScript

6.2 Real-World Impact

- **Samy Worm (MySpace, 2005)** — 1 million profiles infected in 20 hours
- **British Airways (2018)** — 380,000 customers' card data stolen via injected script

- **OWASP Ranking:** A03:2021 Injection

6.3 Scenario

A fake comment board is used. Users post comments that all visitors see. The attacker posts a comment containing JavaScript. Every subsequent visitor who loads the board has the script execute in their browser.

6.4 Vulnerable Code

```
# app.py - /xss/vulnerable (DELIBERATELY INSECURE)
author = request.form.get('author', '')
content = request.form.get('content', '')

# VULNERABLE: stores raw user input without any sanitisation
conn.execute("INSERT INTO comments (author, content) VALUES (?, ?)",
              (author, content))

// JavaScript - VULNERABLE: innerHTML parses and executes HTML/JS
commentDiv.innerHTML = comment.content; // <script> tags RUN here
```

6.5 Attack Payloads

```
<!-- Proof of concept: alert dialog -->
<script>alert('XSS! You have been hacked!')</script>

<!-- Cookie theft simulation -->
<img src=x onerror="alert('Cookie: ' + document.cookie)">

<!-- Page defacement -->
<b style='color:red;font-size:2rem'>Page Defaced!</b>
```

6.6 Secure Code

```
# app.py - /xss/secure (FIXED)
import html

author = html.escape(request.form.get('author', ''))
content = html.escape(request.form.get('content', ''))
# <script> becomes &lt;script&gt; - displayed, never executed

conn.execute("INSERT INTO comments (author, content) VALUES (?, ?)",
              (author, content))

// JavaScript - SECURE: textContent treats everything as plain text
commentDiv.textContent = comment.content; // no HTML parsing
```

What `html.escape()` converts:

Input	Escaped Output
<	<
>	>
"	"
'	'
&	&

6.7 Comparison

	Vulnerable	Secure
Storage	Raw content saved	<code>html.escape()</code> applied
Rendering	<code>innerHTML</code> (pares HTML)	<code>textContent</code> (plain text)
<code><script></code>	Executes in browser	Shown as text <code>&lt;script&gt;</code>

6.8 Prevention Checklist

- Use `html.escape()` (Python) on all user-supplied output
- Use `textContent` instead of `innerHTML` in JavaScript
- Implement Content Security Policy (CSP) headers
- Jinja2's `{{ var }}` auto-escapes — never use `{{ var | safe }}` with user data
- **Never** use `innerHTML` with untrusted content

7 Attack 3 — Parameter Tampering

7.1 Concept

Parameter Tampering exploits the fact that web applications often pass business-critical values — prices, discounts, user IDs, roles — as hidden form fields, URL query strings, or cookies, trusting the client not to modify them. An attacker uses browser DevTools or an intercepting proxy to alter these values before submission.

7.2 Real-World Impact

- E-commerce platforms regularly lose revenue to price tampering
- Privilege escalation: changing `role=user` to `role=admin` in a URL parameter
- **OWASP Ranking:** A04:2021 Insecure Design

7.3 Scenario

A fake e-commerce shop is used. Products have prices stored in the database. The page renders a hidden form field with the product price. When the user clicks Buy, the server reads the price from this field — which the attacker modifies using DevTools before submitting.

7.4 Vulnerable Code

```
# app.py - /parameter-tampering/vulnerable (DELIBERATELY INSECURE)
product_id = request.form.get('product_id', type=int)
product_name = request.form.get('product_name', '')

# VULNERABLE: reads price from client - attacker controls this!
paid_price = request.form.get('price', type=float, default=0.0)

conn.execute(
    "INSERT INTO orders (product_name, real_price, paid_price) VALUES (?, ?, ?)",
    (product_name, real_price, paid_price)
)
```

7.5 Attack Steps

1. Select **iPhone 16 Pro (Rs. 1,29,999)** — page renders hidden field:
value="129999"
2. Open **DevTools (F12)** → Inspector → find the hidden price input
3. Edit: change value="129999" to value="1"
4. Click **Place Order** — server receives paid_price = 1
5. Server trusts the client value and records the order at **Rs. 1**

7.6 Secure Code

```
# app.py - /parameter-tampering/secure (FIXED)
product_id = request.form.get('product_id', type=int)
# Client-supplied price is completely IGNORED

# SECURE: fetch authoritative price from the database
row = conn.execute(
    "SELECT price FROM products WHERE id=?", (product_id,)
).fetchone()
server_price = row['price'] # DB is the single source of truth

conn.execute(
    "INSERT INTO orders (product_name, real_price, paid_price) VALUES (?, ?, ?)",
    (product_name, server_price, server_price)
)
```

7.7 Comparison

	Vulnerable	Secure
Price source	request.form['price'] (client)	DB lookup by product ID
Client control	Client dictates the price	Client value ignored

	Vulnerable	Secure
Result	iPhone bought for Rs. 1	Real price always charged

7.8 Prevention Checklist

- Never trust client-supplied prices, discounts, or quantities
- Always fetch authoritative values from the server-side database
- Validate all parameters: type, range, whitelist of allowed values
- Sign or HMAC-encrypt sensitive parameters if they must travel client-side
- **Never** use hidden form fields to store business-critical values like price

8 Application Features

Feature	Description
Dual-mode per attack	Toggle between Vulnerable and Secure mode on the same page
Live terminal output	Shows exact SQL query or server response in real time
Payload chips	One-click auto-fill of attack payloads for easy demo
Code diff	Side-by-side before/after code shown on every attack page
Order log	Persistent table of all orders with TAMPERED / OK badges
Comment board	XSS scripts execute live in vulnerable mode
Reset button	XSS comment board can be reset to seed data
Toast notifications	Instant feedback on every form action

9 Results and Observations

Attack	Vulnerable Mode	Secure Mode
SQL Injection	admin' -- bypasses login. Admin account with Rs. 98,500 balance exposed without knowing the password.	Login fails. Parameterized query rejects the payload.
XSS	<script>alert()</script> executes in browser. onerror accesses document.cookie.	Script rendered as plain text. html.escape() neutralises the payload.
Parameter Tampering	iPhone (Rs. 1,29,999) purchased for Rs. 1. Server trusts client price field.	Server fetches DB price. Client value ignored. Real price always charged.

10 Conclusion

Web security vulnerabilities remain among the most prevalent and costly threats in software. Through this project, we demonstrated that each attack is eliminated by a targeted, minimal code change:

1. **SQL Injection** — Parameterized queries treat input as data, never code
2. **XSS** — `html.escape()` and `textContent` instead of `innerHTML`
3. **Parameter Tampering** — Server-side price lookup from the database; client value ignored

“The client is always the enemy. Validate, sanitise, and verify everything on the server.”

11 References

1. OWASP Top 10 (2021) — <https://owasp.org/www-project-top-ten/>
2. OWASP SQL Injection — https://owasp.org/www-community/attacks/SQL_Injection
3. OWASP Cross-Site Scripting — <https://owasp.org/www-community/attacks/xss/>
4. Flask Documentation — <https://flask.palletsprojects.com/>
5. Python `html.escape()` — <https://docs.python.org/3/library/html.html>
6. Python `sqlite3` — <https://docs.python.org/3/library/sqlite3.html>
7. Heartland Payment Breach (2008) — Krebs on Security
8. Samy Worm Analysis (2005) — <https://samy.pl/myspace/tech.html>